MEDICAL STORE MANAGEMENT SYSTEM

INTERNSHIP REPORT

Submitted by

NANDHAKUMAR D
(2303717610421102)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE – 641014

ANNA UNIVERSITY:CHENNAI 600 025

JUNE 2025

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government aided Autonomous Institution Affiliated to Anna University)

COIMBATORE – 641014

ANNA UNIVERSITY::CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this Internship report is the bonafide work of **NANDHAKUMAR D(2303717610421102).**

Dr.A.N.SENTHILVEL M.S., Ph.D

Professor

Department of Computer Science and Engineering,

Coimbatore Institute of Technology, Coimbatore – 641 014.

Dr.A.KUNTHAVAI M.S.,Ph.D

Professor and Head

Department of Computer Science and Engineering,

Coimbatore Institute of Technology, Coimbatore – 641 014.

Submitted for the evaluation held on

Reviewer 1

Reviewer 2

Reviewer 3

DETAILS OF INTERNSHIP UNDERGONE

Reg. No: 2303717610421102

Name of the Student: Nandhakumar D

Degree: B.E.

Branch: Computer Science and Engineering

Semester: IV

Project Title : MEDICAL STORE MANAGEMENT SYSTEM

S. No.	Name and Address of the Company	Period of Internship undergone		No. of Days
		From	To	
1.	**PROJECT DEVELOPMENT CELL (PDC),** DEPARTMENT OF CSE , COIMBATORE INSTITUTE OF TECHNOLOGY, COIMBATORE	19.05.2025	02.06.2025	15
Total Number of Days:				15

Signature of Student **Signature of Tutor**

COPY OF CERTIFICATE:

COIMBATORE INSTITUTE OF TECHNOLOGY

(GOVERNMENT AIDED AUTONOMOUS INSTITUTION AFFILIATED TO ANNA UNIVERSITY)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

OF INTERNSHIP

This certificate is proudly presented to

NANDHAKUMAR D, CSE , CIT

for successfully completing the 15-day Summer Internship 2025 organised by the Project Development Cell (PDC), Department of Computer Science and Engineering, Coimbatore Institute of Technology, Coimbatore. He/ She has performed Excellent/Good/Satisfactory during the internship period from 19/05/2025 to 02/06/2025.

Dr. A. Kunthavai
Professor & Head
Department of CSE, CIT

Dr. A. Rajeswari
Principal, CIT

ACKNOWLEDGEMENT

I extend my heartfelt gratitude to **Project Development Cell (PDC)** for permitting me to undergo internship their industries/organizations and their guidance to upgrade my theoretical and practical skills in engineering.

I am sincerely grateful to the Management of Coimbatore Institute of Technology, Coimbatore for supporting me in every possible way, for undergoing internship in industries.

I sincerely thank our Principal **Dr. A. Rajeswari, M.E., Ph.D.,** Coimbatore Institute of Technology, Coimbatore and Head of the Department **Dr Kunthavai A, M.S., Ph.D.,** Department of Computer Science and Engineering, Coimbatore Institute of Technology, Coimbatore for including industrial training pattern in the curriculum to enrich students with a sound knowledge in engineering and make them industry ready. I am also obliged for permitting me to undergo the training.

I also extend my thanks to the Senior Tutor, Tutors, Teaching faculty and all other supporting staff members of the department for their support and encouragement to take up the internship in industries.

NANDHAKUMAR D

TABLE OF CONTENTS

LIST OF ABBREVIATION

ABBREVIATION	EXPANSION
API	Application Programming Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DB	DataBase
UI	User Interface
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
URL	Uniform Resource Locator
CRUD	Create,Read,Update,Delete

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

1st WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED
	19.05.2025	MONDAY	Introduction to HTML & CSS
	20.05.2025	TUESDAY	Javascript,PostgreSQL
	21.05.2025	WEDNESDAY	Introduction to Full Stack Development with FastAPI
	22.05.2025	THURSDAY	Project Planning – Module Breakdown and Flowchart
	23.05.2025	FRIDAY	User Authentication – Admin & Staff Login Module (FastAPI + HTML)
	24.05.2025	SATURDAY	Medicine Inventory Module – Add/Update/Delete Medicines with psycopg2
	25.05.2025	SUNDAY	UI Improvements and CSS Styling for Forms

2nd WEEK	DATE	DAY	NAME OF THE TOPIC/MODULE COMPLETED
	26.05.2025	MONDAY	Billing Module – Sales Entry & Automatic Invoice Generation
	27.05.2025	TUESADAY	Supplier Management Module – Add/Update/Delete Supplier Records
	28.05.2025	WEDNESDAY	Expiry Tracking & Low Stock Alert Feature Implementation
	29.05.2025	THURSDAY	Admin Dashboard – Staff, Supplier & Stock Management
	30.05.2025	FRIDAY	Reporting Module – Daily/Monthly Sales & Profit Reports
	31.05.2025	SATURDAY	Bug Fixing, Code Cleanup, and UI Finalization
	01.06.2025	SUNDAY	Documentation – SRS, Use Cases, and Report Drafting
	02.06.2025	MONDAY	Final Testing and Submission of Completed Project

CHAPTER 1

INTRODUCTION

The Medical Store Management System offers an efficient and automated approach to managing pharmacy operations, addressing the limitations of traditional manual processes. It streamlines tasks such as inventory tracking, sales recording, billing, supplier management, and expiry monitoring within a unified and user-friendly platform. Designed to reduce human error and enhance accuracy, the system ensures timely decision-making through real-time updates and alerts. By simplifying daily operations and improving reliability, it enables pharmacies to maintain better control over stock, ensure regulatory compliance, and provide a smoother service experience to customers..

1.1 PROJECT SCOPE:

This project involves designing and deploying a medical store management system that streamlines pharmacy operations through automation, accuracy, and secure access.

- **Inventory Management:** Enables adding, updating, and tracking medicines with expiry and low-stock alerts.
- **Sales and Billing:** Supports fast and error-free billing with auto-filled medicine details and invoice generation.
- **Supplier and Purchase Tracking:** Maintains supplier records and purchase history for efficient restocking.
- **User Access Control:** Provides role-based authentication for admin and staff to ensure security and data integrity.
- **Reporting and Analytics:** Generates detailed reports on sales, inventory, and expired stock for better decision-making.
- **Applications:** Useful for pharmacies, clinics, and hospital dispensaries to ensure smooth medicine management.

OBJECTIVES:

The main objectives of the project are:
• To design and develop an efficient medical store management system that automates key pharmacy operations.
• To enable accurate tracking and management of medicine inventory, including stock levels and expiry dates.
• To provide quick and reliable sales and billing functionality with automatic invoice generation.
• To maintain detailed records of suppliers and purchase transactions for effective restocking.
• To offer a user-friendly interface for managing medicines, handling billing, and generating reports.
• To implement secure login with role-based access control for admins and staff.

1.2 MODULES:

1.2.1 User Authentication Module

Description: This module handles secure login and session management for system users. It restricts access based on user roles like admin or staff.

Functionality:
• Allows users to log in using valid credentials.
• Verifies username and password against the database.
• Sets a secure session cookie using a serializer.
• Redirects to a role-based dashboard upon successful login.

1.2.2 Medicine Inventory Module

Description: This module manages all operations related to medicines, including adding new stock, viewing current inventory, and tracking expiry dates.

Functionality:
• Add or update medicine details such as name, price, quantity, expiry date, and batch.
• Fetch and display the list of medicines through a dedicated API endpoint.
• Show total medicines on the dashboard.
• Automatically update quantities when medicines are sold or added.

1.2.3 Sales and Billing Module

Description: This module handles billing operations and records all customer purchases with medicine and transaction details.

Functionality:

• Collect customer details and selected medicines via a billing form.
• Calculate total price and generate a bill.
• Insert billing information into the bills table.
• Deduct sold quantity from stock and record sales in the sales table.
• Display individual or recent billing records for customer review.

1.2.4 Reporting and Analytics Module

Description: This module generates real-time insights on inventory, sales, and earnings to support business decisions**.**

Functionality:

• Compute totals for medicine quantity, sales, and arrival records.
• Generate daily earnings and transaction summaries.
• Show trends using grouped sales and arrivals data.
• Present reports on a structured dashboard with historical insights.

1.2.5 Supplier and Purchase Module

Description: This module maintains supplier information and tracks the arrival of medicine stock into the store**.**

Functionality:

• Track quantity of medicines arrived over time using the arrivals table.
• Aggregate daily arrival data for reporting purposes.
• Supports linking supplier records with restocked inventory.

1.2.6 Billing History Module

Description: This module provides access to the complete billing record history for review, printing, and audit.

Functionality:

• Retrieve and display a list of all bills stored in the system.
• Provide easy access to detailed billing data per transaction.
• Allow filtering or browsing by bill ID or date (expandable).

CHAPTER 2

SOFTWARE REQUIREMENTS SPECIFICATIONS

2.1 Functional Requirements

User Authentication

- Admin and staff can log in securely.
- Login/logout functionality based on user roles.
- Password reset handled by admin or security procedures.

Role-Based Dashboards

- Staff can manage medicines, generate bills, and view reports.
- Admin can manage staff accounts, suppliers, and view full system reports.

Medicine Management

- Add, update, or remove medicines.
- Track quantity, price, and expiry date.
- Automatically update inventory when sales or arrivals are processed.

Billing and Sales

- Generate bills for customers with auto-calculated totals.
- Update medicine stock after billing.
- Record all sales in the database in real time.
- Generate invoices with unique invoice IDs.

Arrival/Stock Management

- Record incoming stock with quantity, date, and supplier info.
- Update medicine inventory automatically after arrivals.

Reporting

- Generate daily, weekly, or monthly reports for sales, inventory, and arrivals.
- Calculate total sales, quantity sold, and revenue earned.
- Admin can export reports for analysis.

Database Management

- PostgreSQL database stores medicines, sales, arrivals, staff, and billing information.
- Enforce unique constraints where necessary (e.g., medicine ID, invoice ID).

2.2 Non-Functional Requirements

Performance

- System should support multiple staff users performing tasks simultaneously.
- Operations like login, billing, and report generation respond within 2 seconds.

Security

- Role-based access to protect restricted pages (admin-only pages vs staff pages).
- Data in the database should be protected against unauthorized access.

Scalability

- Database and application can handle increasing medicines, customers, and sales records.

Usability

- User-friendly interface for staff and admin.
- Compatible with modern browsers on desktops and tablets.

Reliability

- Database ensures consistency and integrity.
- Transaction rollbacks in case of errors during billing or stock updates.

Maintainability

- Modular code with reusable components.
- Configuration (e.g., database credentials) handled via environment variables.

2.3 Software Requirements

- **Operating System:** Windows/Linux
- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python (FastAPI, Pydantic, psycopg2)
- **Database:** PostgreSQL
-

2.4 Hardware Requirements

- **Processor:** Intel i3 or above
- **RAM:** 4 GB minimum
- **Storage:** 250 MB for local database setup

CHAPTER-3

SYSTEM ARCHITECTURE AND DESIGN

3.1. System Design

The system is designed using a modular, layered architecture.

- **Presentation Layer (UI Layer):**
  Provides forms and dashboards for Admin and Staff. Handles input validation before sending data to the backend.
- **Application Layer (Business Logic):**
  Contains the core logic for adding, updating, deleting, and viewing medicines.
  Manages inventory operations, billing, supplier details, and report generation.
  Handles alerts for expiry dates and low stock.
- **Database Layer (Data Storage):**
  Stores user data, medicine details, supplier records, billing information, and feedback.
  Supports CRUD (Create, Read, Update, Delete) operations.

3.2. System Architecture

3.2.1 Architectural Style

- Follows a **3-Tier Architecture**:
  1. **Presentation Layer** – User Interface (UI forms, dashboards).
  2. **Application Layer** – Business logic (Python/FastAPI backend).
  3. **Data Layer** – Database (PostgreSQL/MySQL/SQLite).

3.2.2 Architecture Components

1. **User Interface Module**
   - **Admin Dashboard:** Add/Update/Delete medicines, Manage Suppliers, Generate Reports, Manage Feedback.

- **Staff Dashboard:** Handle Sales & Billing, View Medicine Inventory, Provide Feedback.

2. **Medicine Management Module**
   - Add new medicine.
   - Update medicine details.
   - Delete expired/outdated medicine.
   - View medicine stock list with expiry and quantity details.
3. **User Management Module**
   - Admin login with role-based access.
   - Staff login for handling daily operations.
   - Profile management for users.
4. **Billing & Sales Module**
   - Generate bills for sales.
   - Maintain sales history with date, medicine, and customer details.
   - Automatic invoice generation.
5. **Supplier & Purchase Module**
   - Add and manage suppliers.
   - Record purchase details for stock restocking.
6. **Feedback Management Module**
   - Submit feedback (by staff).
   - Manage and review feedback (by admin).
7. **Database**
   - Tables for Users, Medicines, Suppliers, Sales, Purchases, Feedbacks.

3.3 ARCHITECTURE DIAGRAM

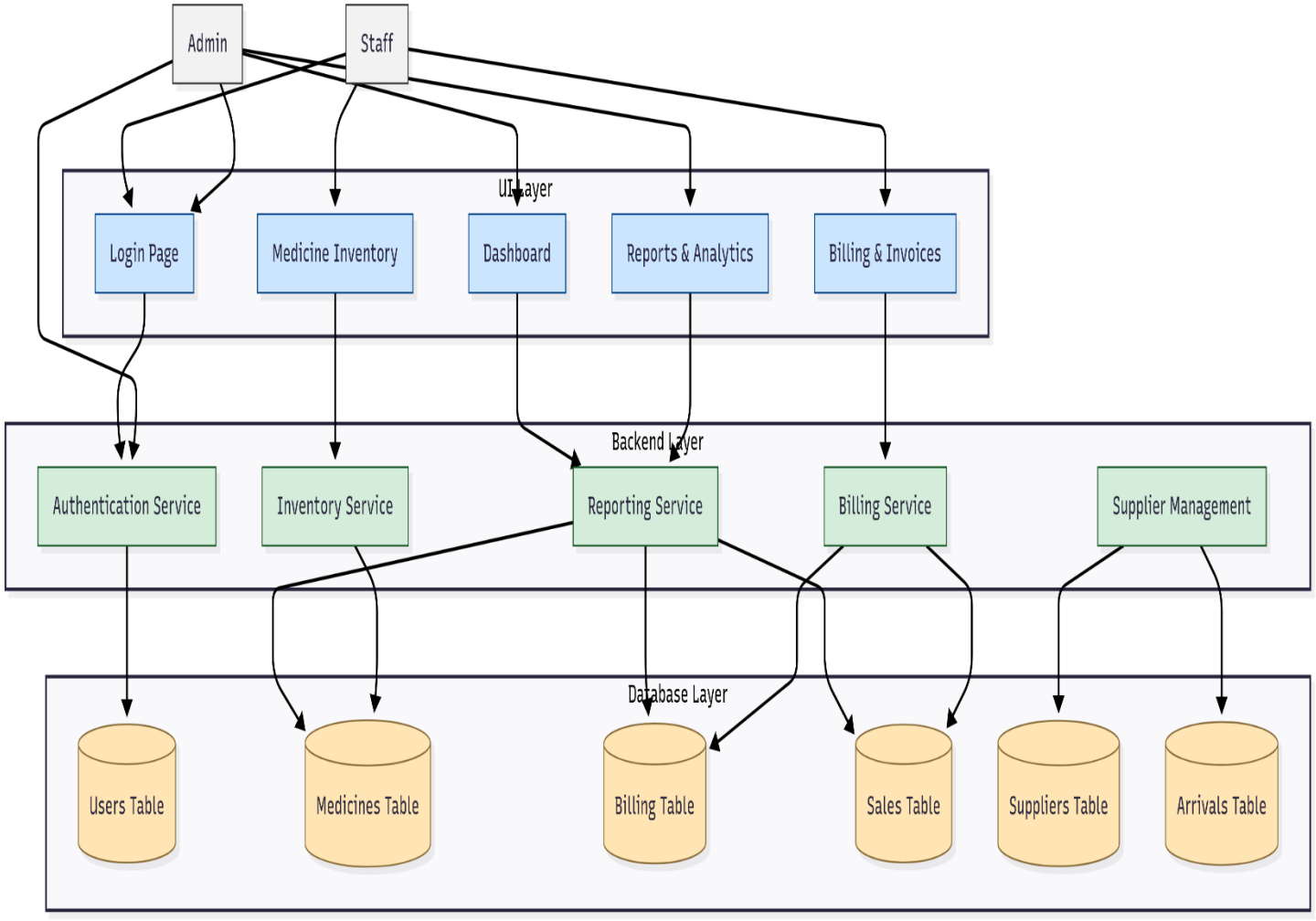

Fig.3.3 Architecture diagram

3.4 WORK FLOW DIAGRAM

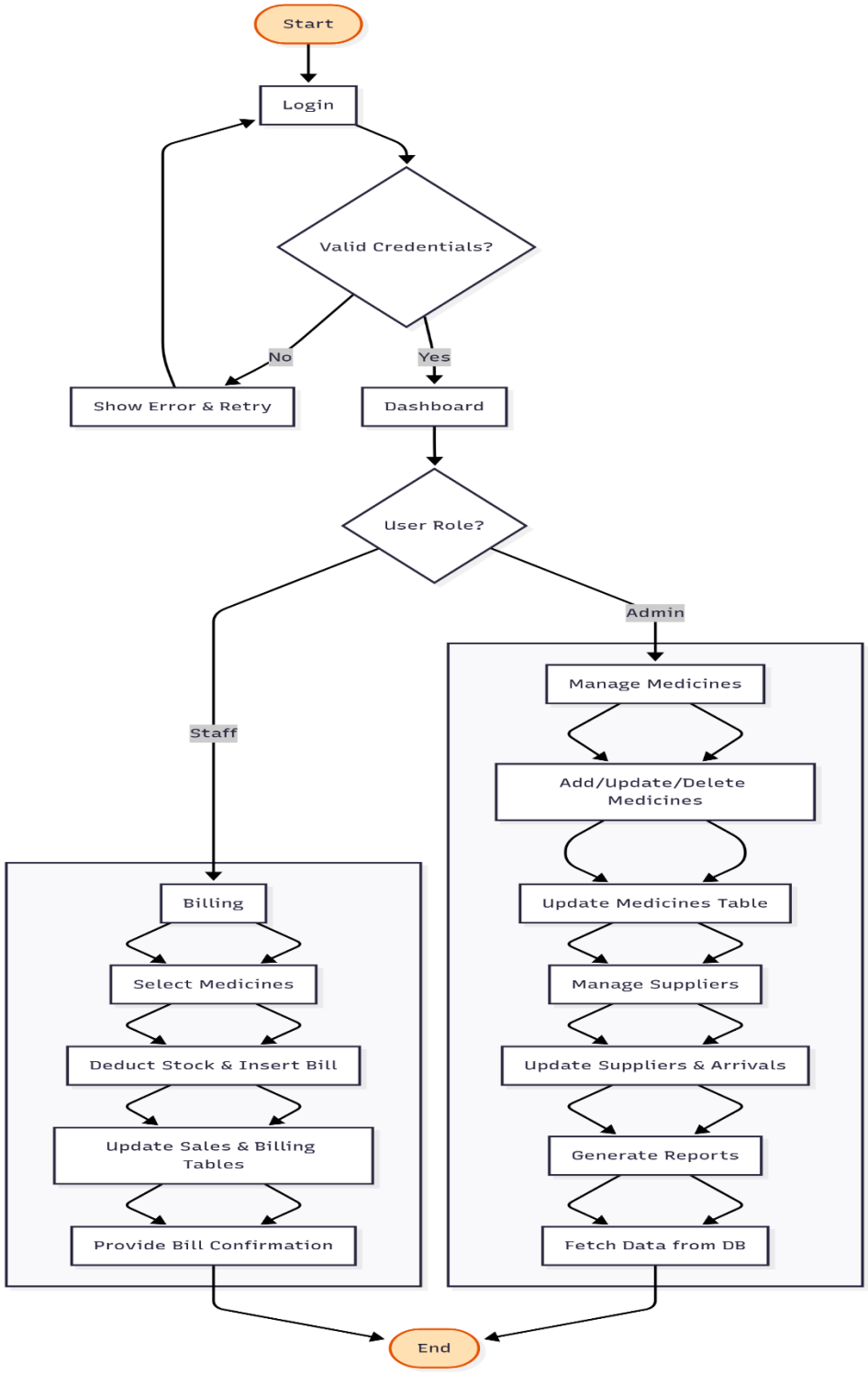

Fig 3.4 Work flow diagram

CHAPTER-4

TECHNOLOGY

4.1. **Frontend Technology**

- **HTTP Request & Response:** Used for communication between client (UI/Postman) and server (FastAPI).
- **Postman (Testing Tool):** Used as a frontend client for sending requests (Add/Update/Delete/View Stocks, Feedbacks) to the backend APIs and testing the responses.

4.2.**Backend Technology**

- **Python:** Core programming language for business logic.
- **FastAPI:** Framework for building APIs quickly and efficiently. It handles request routing, response generation, and integrates well with databases.
- **PostgreSQL:** Relational Database Management System used for storing medicines, users, prescriptions, and feedback details.

4.3 Supporting Tools & Technologies

- **HTTP Methods:**
  - **POST:** Add new medicine, register user, add feedback.
  - **PUT/PATCH:** Update medicine details.
  - **DELETE:** Remove medicine records.
  - **GET:** Retrieve medicine lists, prescriptions, and feedback.
- **Database Connector (ORM/Driver):** Used for connecting FastAPI with PostgreSQL (e.g., SQLAlchemy, psycopg2).
- **JSON:** Data format for request/response between frontend (Postman/Client) and backend.

CHAPTER-5
CODING

1.main.py

```
from fastapi import FastAPI, Request, Form, Response, Cookie, HTTPException
from fastapi.responses import HTMLResponse, RedirectResponse, JSONResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
import psycopg2
from psycopg2.extras import RealDictCursor
from pydantic import BaseModel
from collections import defaultdict
from datetime import datetime, date
from itsdangerous import URLSafeSerializer
from typing import Optional, List
import time
import traceback
app = FastAPI()
app.mount("/static", StaticFiles(directory="static"), name="static")
serializer = URLSafeSerializer("MY_SECRET_KEY")
templates =
Jinja2Templates(directory=r"C:\Users\crazy\OneDrive\Documents\Desktop\summerintern\temp
lates")
# PostgreSQL connection
pg_conn = None
while True:
    try:
        pg_conn = psycopg2.connect(
```

```
            host='localhost',
            database='database1',
            user='postgres',
            password='nandha102',
            cursor_factory=RealDictCursor
        )
        print("✅ PostgreSQL connected")
        break
    except Exception as e:
        print("❌ PostgreSQL connection error:", e)
        time.sleep(3)
# Models
class Item(BaseModel):
    medId: int
    name: str
    qty: int
    price: float
    total: float
class BillingData(BaseModel):
    user_id: str
    customer_name: str
    mobile_number: str
    district: str
    items: List[Item]
    add_date: str
# Routes
```

```
@app.get("/login", response_class=HTMLResponse)
def login_form(request: Request):
    return templates.TemplateResponse("login.html", {"request": request})
@app.post("/login", response_class=HTMLResponse)
def login(request: Request, response: Response, username: str = Form(...), password: str =
Form(...)):
    try:
        cur = pg_conn.cursor()
        cur.execute("SELECT * FROM users WHERE name = %s", (username,))
        user = cur.fetchone()
        cur.close()
        if not user:
            return templates.TemplateResponse("login.html", {"request": request, "error": "User not
found"})
        if user["password"] != password:
            return templates.TemplateResponse("login.html", {"request": request, "error": "Incorrect
password"})
        session_data = serializer.dumps({"username": user["name"], "role": user["role"]})
        resp = RedirectResponse(url="/dashboard", status_code=303)
        resp.set_cookie(key="session", value=session_data, httponly=True)
        return resp
    except Exception:
        print("❌ Exception during login:")
        print(traceback.format_exc())
        return templates.TemplateResponse("login.html", {"request": request, "error": "Login error.
Try again."})
@app.get("/dashboard", response_class=HTMLResponse)
```

```
def dashboard(request: Request):
    try:
        cur = pg_conn.cursor()
        cur.execute("SELECT COUNT(*) AS count FROM medicines")
        tot_med = cur.fetchone()["count"]
        cur.execute("SELECT COUNT(*) AS count FROM sales")
        tot_sales = cur.fetchone()["count"]
        cur.execute("SELECT COUNT(*) AS count FROM arrivals")
        tot_arrivals = cur.fetchone()["count"]
        cur.close()
        return templates.TemplateResponse("dashboard.html", {
            "request": request,
            "medicines": tot_med,
            "sales": tot_sales,
            "arrivals": tot_arrivals})
    except Exception:
        return HTMLResponse(f"<h1>Server
Error:<br><pre>{traceback.format_exc()}</pre></h1>", status_code=500)
@app.get("/medicines", response_class=HTMLResponse)
def render_medicines(request: Request):
    return templates.TemplateResponse("medicines.html", {"request": request})
@app.get("/api/medicines")
def get_medicines():
    try:
        cur = pg_conn.cursor()
        cur.execute("""
            SELECT name, expiry_date, quantity, price
```

```
            FROM medicines
            WHERE expiry_date >= CURRENT_DATE """)
        data = cur.fetchall()
        cur.close()
        return {"medicines": [[row["name"], row["expiry_date"], row["quantity"], row["price"]]
for row in data]}
    except Exception as e:
        return {"error": str(e)}
class Medicine(BaseModel):
    id: int | None = None
    name: str
    expiry_date: str
    quantity: int
    price: float
    added_date: str | None = None
@app.get("/add_medicine", response_class=HTMLResponse)
def render_add_medicine_form(request: Request):
    return templates.TemplateResponse("add_medicine.html", {"request": request})
# 👇 This matches frontend's fetch('/add_med')
@app.post("/add_med")
def add_medicine(med: Medicine):
    try:
        cur = pg_conn.cursor()
        if med.id is None:
            cur.execute("""
                INSERT INTO medicines (name, expiry_date, quantity, price, added_date)
                VALUES (%s, %s, %s, %s, %s)
```

```
            RETURNING id
        """, (med.name, med.expiry_date, med.quantity, med.price, med.added_date or
datetime.now()))
    else:
        cur.execute("""
            INSERT INTO medicines (id, name, expiry_date, quantity, price, added_date)
            VALUES (%s, %s, %s, %s, %s, %s)
            RETURNING id
        """, (med.id, med.name, med.expiry_date, med.quantity, med.price, med.added_date or
datetime.now()))
    new_id = cur.fetchone()["id"]
    pg_conn.commit()
    cur.close()
    return {"message": "Medicine added successfully", "id": new_id}
  except Exception as e:
    raise HTTPException(status_code=500, detail=f"Database error: {str(e)}")
@app.get("/report", response_class=HTMLResponse)
def generate_report(request: Request):
  try:
    cur = pg_conn.cursor()
    cur.execute("SELECT * FROM medicines")
    medicines = cur.fetchall()
    cur.execute("SELECT SUM(quantity) AS sum FROM medicines")
    total_qty = cur.fetchone()["sum"] or 0
    cur.execute("SELECT SUM(quantity_arrived) AS sum FROM arrivals")
    total_arrived = cur.fetchone()["sum"] or 0
    cur.execute("SELECT SUM(quantity_sold) AS sum FROM sales")
```

```
    total_sold = cur.fetchone()["sum"] or 0
    cur.execute("SELECT SUM(quantity_sold * price) AS sum FROM sales")
    total_amount = cur.fetchone()["sum"] or 0.0
    cur.execute("""SELECT sales_date::date as date,SUM(quantity_sold) as sold,
            SUM(quantity_sold * price) as earned FROM sales GROUP BY sales_date::date
       ORDER BY sales_date::date""")
    sales = cur.fetchall()
cur.execute("""SELECT arrival_date::date as date,SUM(quantity_arrived) as arrived
       FROM arrivals GROUP BY arrival_date::date ORDER BY arrival_date::date """)
    arrivals = cur.fetchall()
    report = defaultdict(lambda: {"arrived": 0, "sold": 0, "earned": 0.0})
    for row in arrivals:
       report[row["date"]]["arrived"] = row["arrived"]
    for row in sales:
       report[row["date"]]["sold"] = row["sold"]
       report[row["date"]]["earned"] = float(row["earned"])
    daily_report = [{
       "date": dt,
       "arrived": report[dt]["arrived"],
       "sold": report[dt]["sold"],
       "earned": report[dt]["earned"]
    } for dt in sorted(report)]
    cur.close()
    return templates.TemplateResponse("report.html", {
       "request": request,
       "medicines": medicines,
       "total_qty": total_qty,
```

```
            "total_arrived": total_arrived,
            "total_sold": total_sold,
            "total_amount": total_amount,
            "daily_report": daily_report
        })
    except Exception:
        print("Error in /reports:", traceback.format_exc())
        return HTMLResponse(f"<h1>Error generating
report</h1><pre>{traceback.format_exc()}</pre>", status_code=500)
@app.get("/billing", response_class=HTMLResponse)
def render_billing_form(request: Request):
    return templates.TemplateResponse("billing.html", {"request": request})
            cur.execute("UPDATE medicines SET quantity = quantity - %s WHERE id = %s",
(item.qty, item.medId))
            cur.execute("INSERT INTO sales (med_id, quantity_sold, price, sales_date) VALUES
(%s, %s, %s, %s)", (item.medId, item.qty, item.price, date_obj)
        pg_conn.commit()
        cur.close()
        return HTMLResponse(content=f"""
            <h1>Billing completed for {data.customer_name}</h1>
            <p>Total: ₹ {grand_total:.2f}</p>
            <a href="/billing_details/{data.user_id}">View Bill</a>""")
    except Exception as e:
        pg_conn.rollback()
        print(traceback.format_exc())
        return JSONResponse(content={"error": str(e)}, status_code=500)
# ✅ Updated route to handle both /billing_details and /billing_details/{billing_id}
```

```
@app.get("/billing_details", response_class=HTMLResponse)
@app.get("/billing_details/{billing_id}", response_class=HTMLResponse)
def show_billing_details(request: Request, billing_id: Optional[int] = None):
    try:
        cur = pg_conn.cursor()
        if billing_id:
            cur.execute("SELECT * FROM bills WHERE id = %s", (billing_id,))
        else:
            cur.execute("SELECT * FROM bills ORDER BY date DESC LIMIT 10")
        rows = cur.fetchall()
        cur.close()
    if not rows:
            return templates.TemplateResponse("billing.html", {
                "request": request,
                "error": "Billing record not found." })
        return templates.TemplateResponse("billing_details.html", {
            "request": request,
            "billing_data": rows})
    except Exception:
        traceback.print_exc()
        return templates.TemplateResponse("billing.html", {
            "request": request,
            "error": "Error loading billing details." })
@app.get("/sales", response_class=HTMLResponse)
def daywise_sales(request: Request):
    cur = pg_conn.cursor()
    cur.execute("SELECT * FROM public.sales ORDER BY sales_date ASC;")
```

```
    sales = cur.fetchall()
    cur.close()
    return templates.TemplateResponse("sales.html", {"request": request, "sales": sales})
@app.get("/billing_history", response_class=HTMLResponse)
def show_all_billing(request: Request):
    try:
        cur = pg_conn.cursor()
        cur.execute("SELECT * FROM bills ORDER BY id ASC")
        billing_records = cur.fetchall()
        cur.close()
        return templates.TemplateResponse("billing_history.html", {
            "request": request,
            "records": billing_records})
    except Exception:
        traceback.print_exc()
        return HTMLResponse(content="<h1>Error loading billing history</h1>",
status_code=500)
```

CHAPTER-6
SCREENSHOTS / RESULTS AND DISCUSSION

6.1 Screenshots:

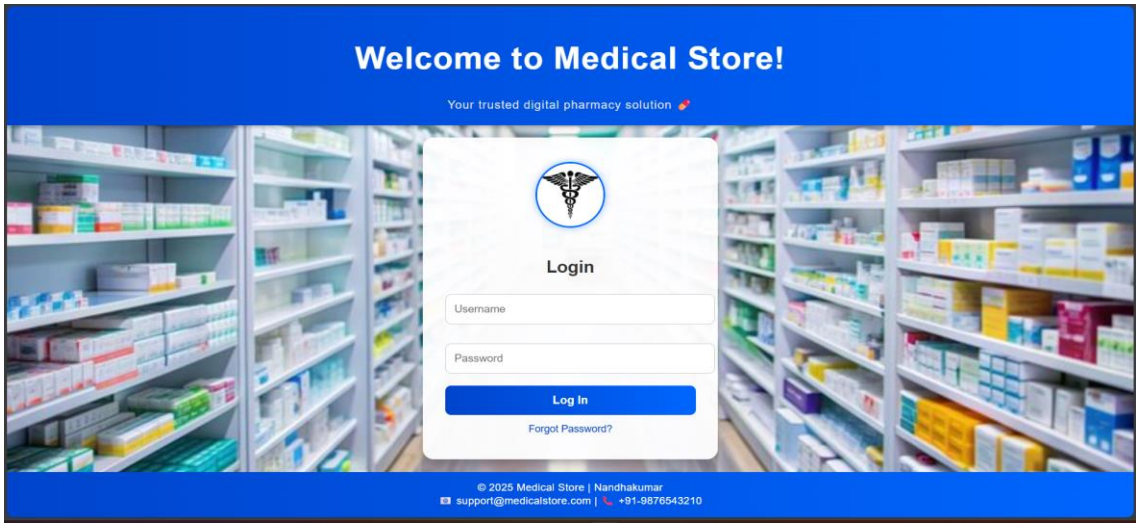

Fig 6.1.1 Login page

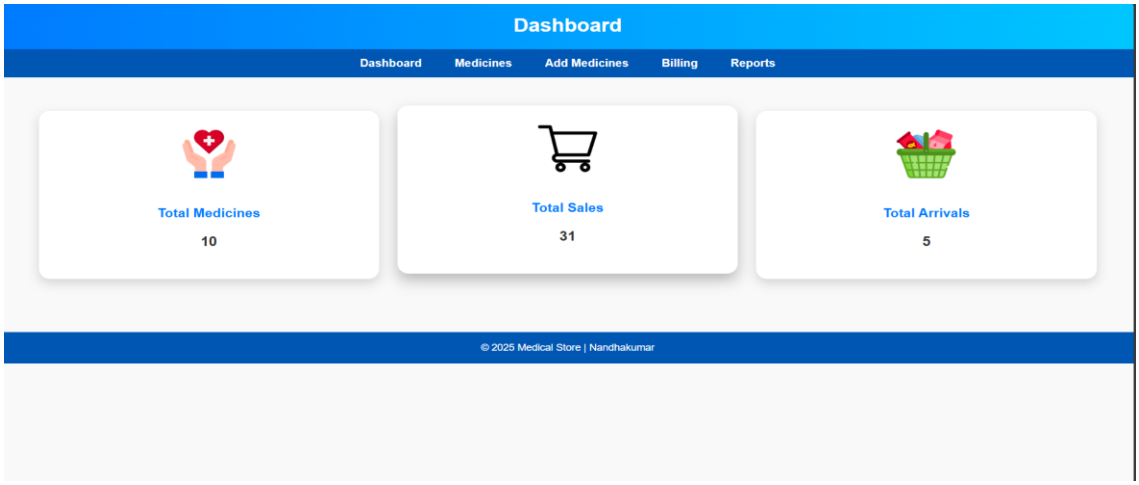

Fig 6.1.2 dashboard page

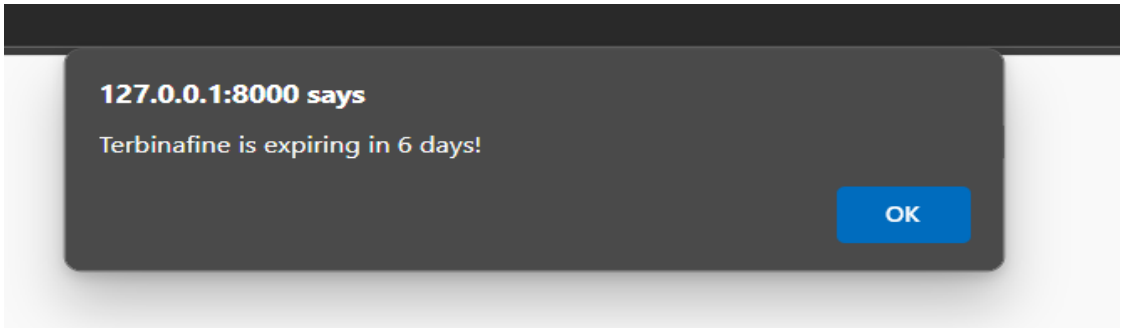

Fig 6.1.3 Expiring medicine alert

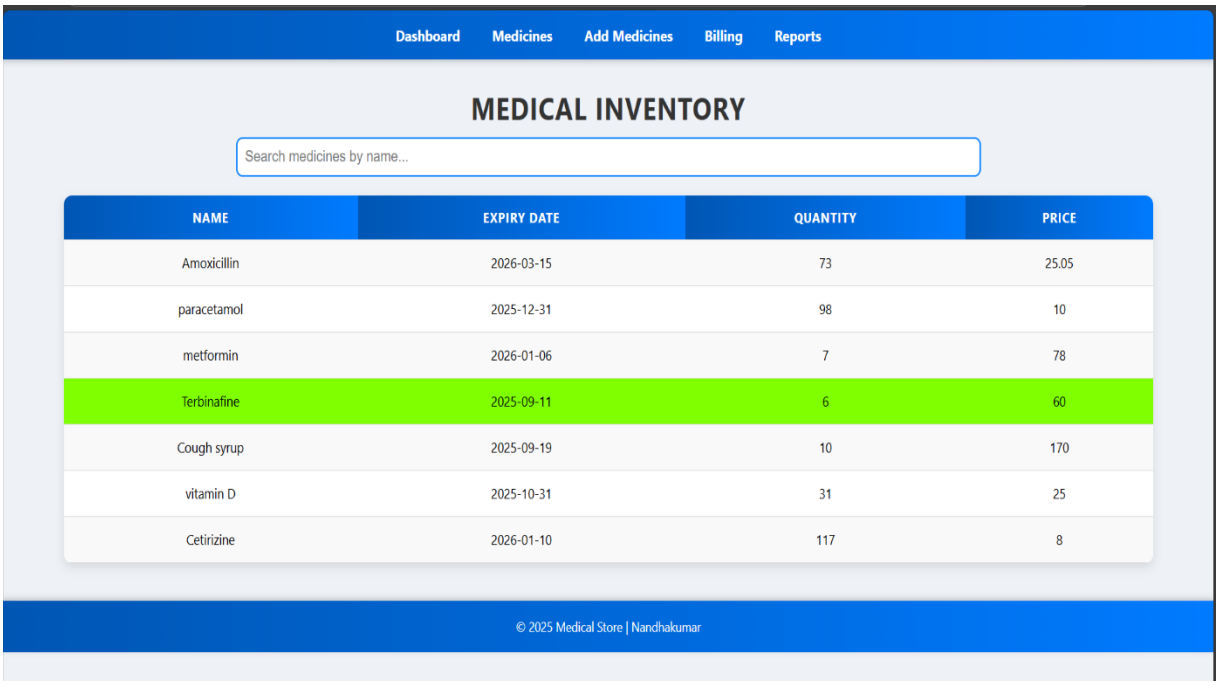

Fig 6.1.4 Medicine Details

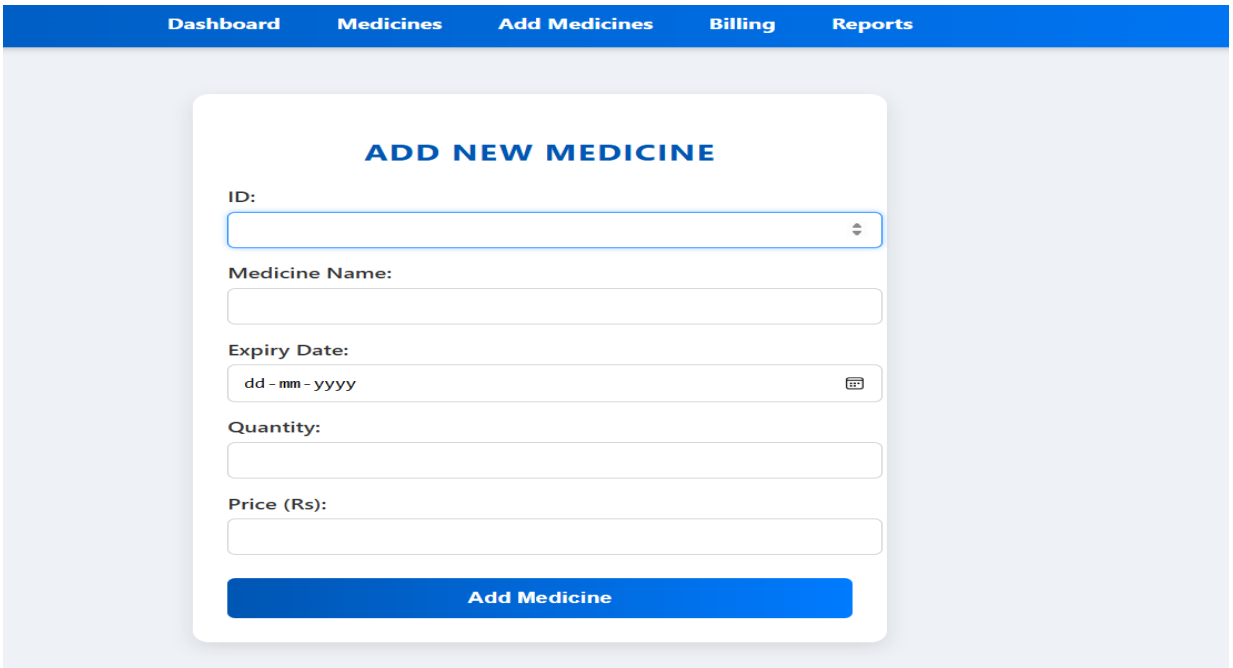

Fig 6.1.5 New Medicine add details

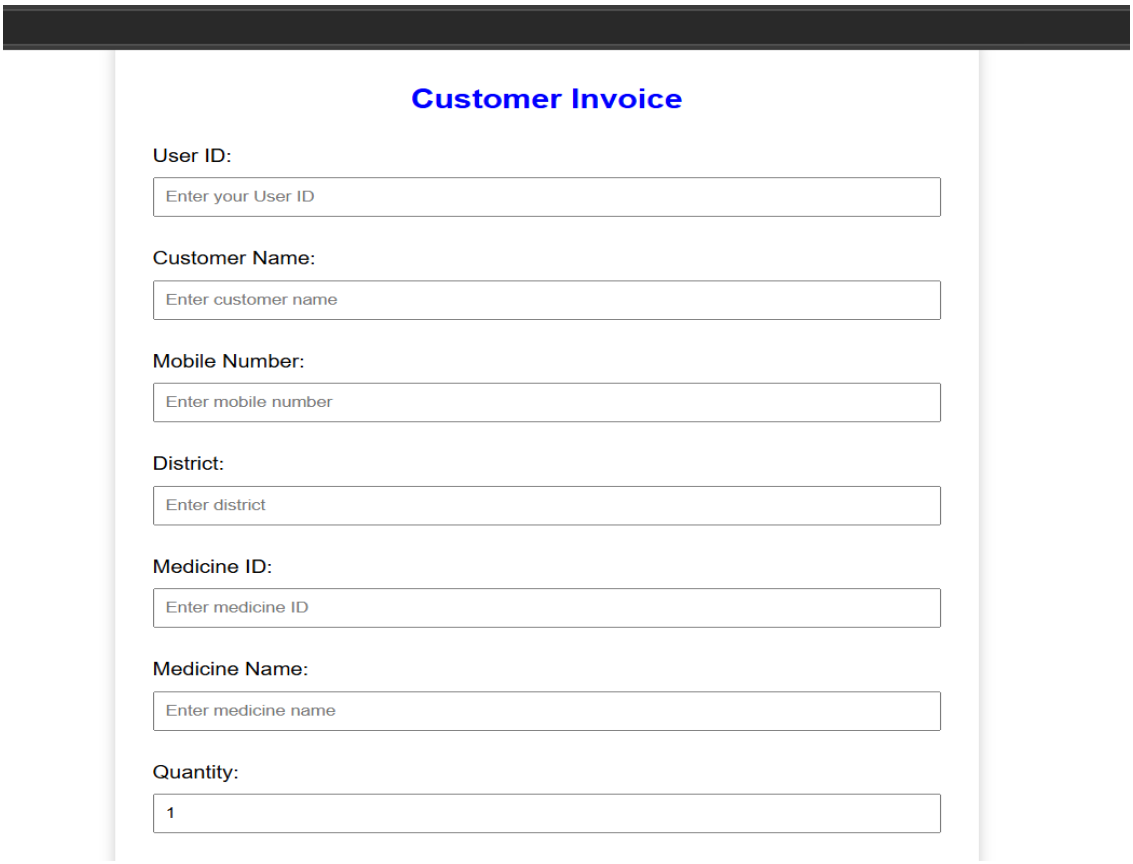

Fig 6.1.6 Billing Form

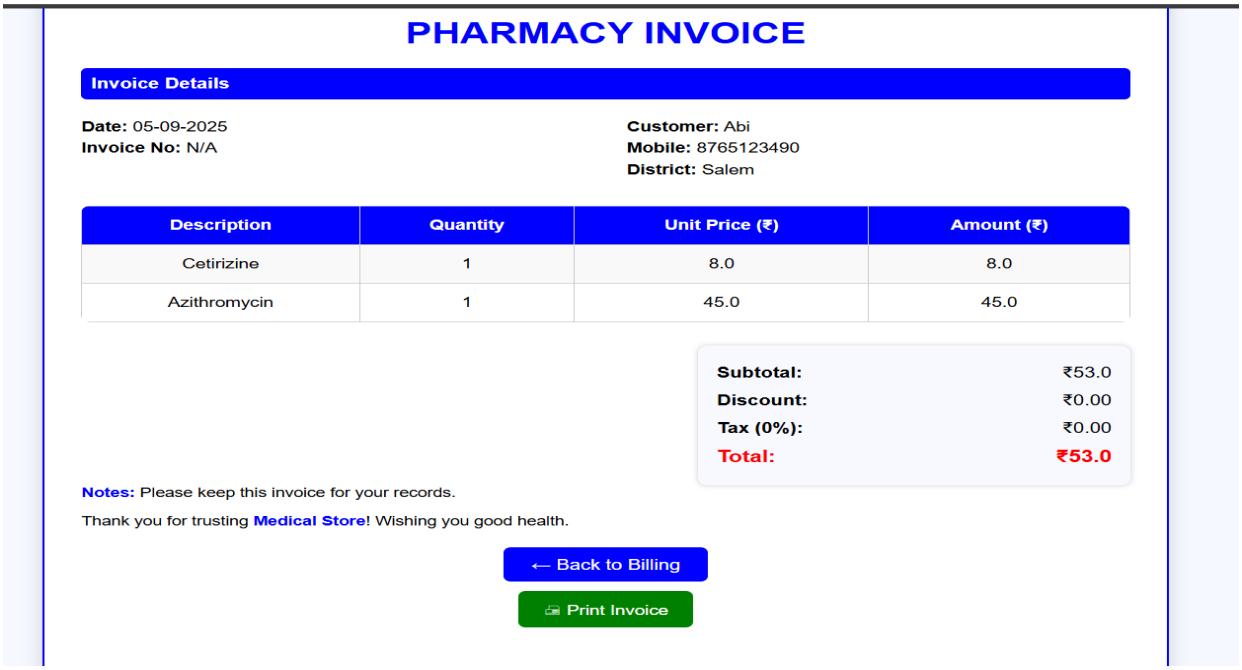

Fig 6.1.7 Billing

Fig 6.1.8 Report page

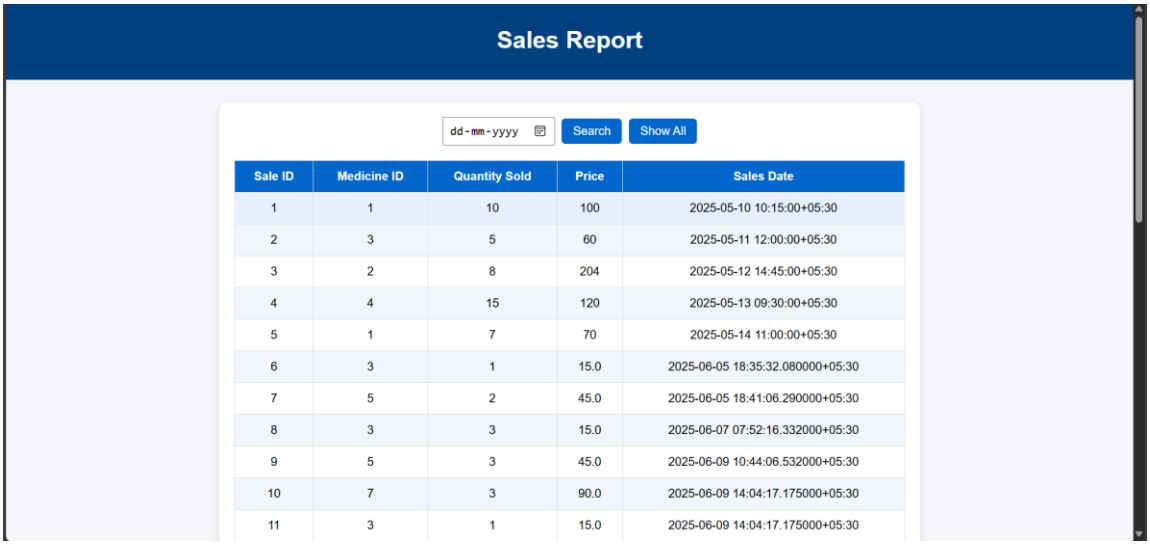

Fig 6.1.9 Daily sales report page

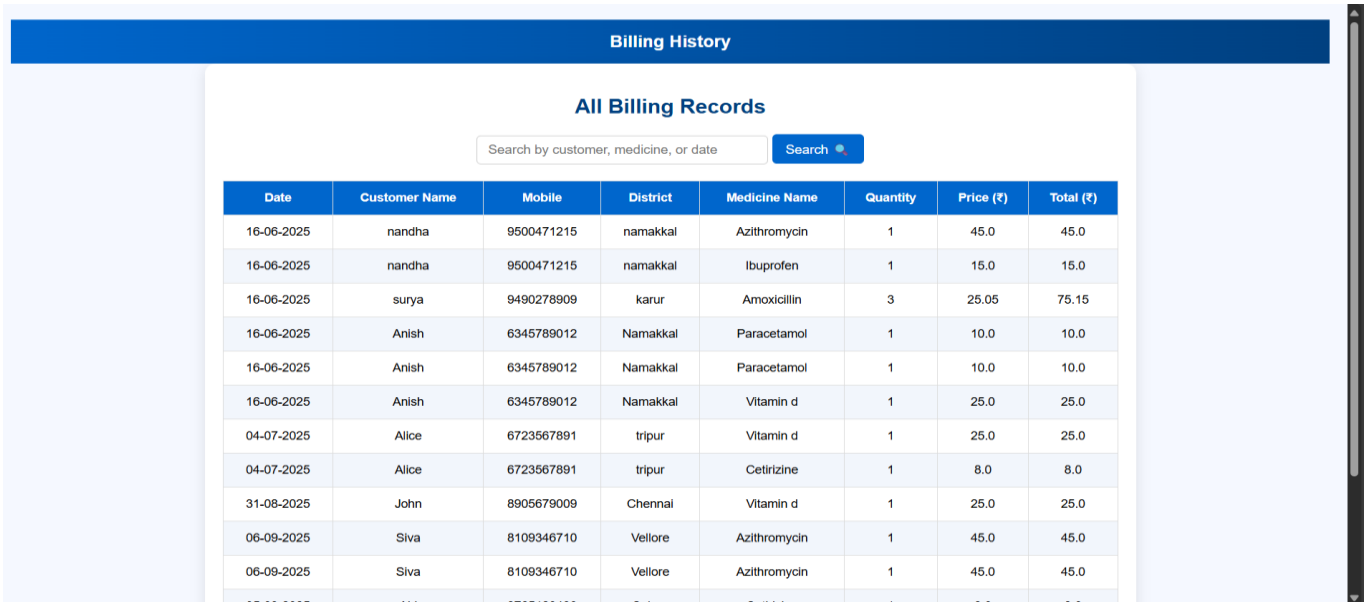

Fig 6.1.10 Daily Billing Report

6.2 Results

The Medical Store Management System was successfully designed and implemented using **[your stack – e.g., Java (backend), MySQL (database), and a web-based frontend with JSP/HTML, CSS, and JavaScript]**. The system provides a **secure, efficient, and user-friendly platform** for managing pharmacy operations.

The major outcomes of the project are as follows:

1. **Accurate Medicine Inventory Management:**
   The system tracks stock levels, expiry dates, and arrivals, ensuring effective medicine management.
2. **Efficient Sales and Billing:**
   Sales transactions are automated with invoice generation and storage in the database for future reference.
3. **Admin Control Panel:**
   Admins can add/remove staff, manage suppliers, and view detailed reports on sales and profits.
4. **Supplier & Purchase Tracking:**
   Supplier details and purchase histories are recorded for smooth restocking and accountability.
5. **Report Generation:**
   The system generates sales, profit, and stock reports for better business insights.
6. **Security and Role-Based Access:**
   Only authenticated users (Admin/Staff) can access the system, ensuring data security.

6.2 Discussions

The developed Medical Store Management System demonstrates the feasibility of **digitally automating pharmacy operations** in a secure and transparent manner. The combination of a relational database and modular backend ensures both **performance and data integrity**.

Advantages observed:

- Improved efficiency compared to manual stock and billing management.
- Reduced human errors in billing and inventory tracking.
- Enhanced accessibility with a simple, user-friendly interface.
- Real-time tracking of medicines, sales, and supplier details.
- Faster decision-making through automated reports.

Challenges faced during development:

- Ensuring strong security for login and access control.
- Designing a simple and intuitive UI for staff with minimal technical knowledge.
- Maintaining accuracy in stock levels and expiry management.
- Handling concurrent billing and inventory updates efficiently.

CHAPTER 7

CONCLUSION

This project successfully designed and implemented a **comprehensive Medical Store Management System** that automates critical pharmacy operations, including medicine inventory, billing, supplier management, and reporting. By integrating **secure authentication, role-based access, and real-time data processing**, the system ensures **accuracy, transparency, and efficiency** in medical store operations.

The modular design ensures scalability and maintainability, with the backend handling core business logic, and the database ensuring reliable storage of medicines, suppliers, billing, and sales data. The system also enables **automatic bill generation, detailed record keeping, and analytical reporting**, thereby reducing manual workload and minimizing errors.

Overall, the system meets its objectives of **streamlining medical store operations, preventing stock mismanagement, supporting secure role-based usage, and enabling data-driven business decisions**. It can be deployed in pharmacies, hospitals, or retail medical stores to improve efficiency and service quality.

CHAPTER-8

BIBLIOGRAPHY

RESEARCH PAPERS:

1. Sakthivel, S., & Kathiresh, M. “Pharmacy Management System: A Comprehensive Solution for Inventory, Billing, and Reporting.” International Journal of Scientific Research and Engineering Development, Vol. 8, Issue 3 (2025).

2. Jibrin A. Dallatu, A. Mai-Abba, Y. Muhammed, & A. Gulani. “RxOptima Management: A Pharmaceutical Management Web Application.” Quest Journals – Journal of Software Engineering and Simulation (2025).

3. Nasri, N., Sentosa, & R. Arvin I. Miracelova. “Application of Pharmacy Management Systems and Digital Marketing: Impact on Operational Efficiency and Turnover.” World Journal of Advanced Research and Reviews, 24(03), (2024).

4. Liang, J. “Optimization of Pharmacy Membership Management Systems through Big Data.” Heliyon – Applied Medical Research (2024).